PROYECTO NANOFILES

Curso 25/26; Convocatoria Enero-Enero
Redes de Comunicaciones
Subgrupo 9.1
Profesor:

Pedro Blaya Pascual (48853636A)
Daniel Pozas Ruiz

ÍNDICE

ÍNDICE 1
1. INTRODUCCIÓN 2
2. PROTOCOLOS DISEÑADOS 2
2.1. Directorio 2
2.1.1 Mensajes y ejemplos 2
2.1.1.1 Tipos y descripción de los mensajes 2
2.1.1.2 Código DirMessageOps 4
2.1.1.3 Código DirMessage 5
2.1.1.4 Código Directory Connector 11
2.1.1.5 Código NFDirectoryServer 20
2.1.2 Autómatas 29
2.1.2.1 Automata peer-directorio 29
2.1.2.2 Autómata directorio-peer 30
2.1.2.3 Codigo automatas 30
2.2. Peers 38
2.2.1 Mensajes y ejemplos 38
2.2.2 Autómatas 39
2.2.2.1 Automata peer-peer-cliente 39
2.2.2.2 Automata peer-peer-servidor 39

1. INTRODUCCIÓN

Este es un primer boceto de nuestra memoria para el proyecto de NanoFiles. En este documento se expone cómo hemos diseñado los distintos mensajes de comunicación, tanto peer-to-peer como un peer con el directorio. También se incluye una representación gráfica de los autómatas con estados finitos. Toda esta implementación de mensajes y autómatas vendrá acompañada con el código correspondiente a su implementación.

En cuanto a las mejoras, hemos considerado la implementación de todas las posibles.

2. PROTOCOLOS DISEÑADOS

2.1. Directorio.

2.1.1 Mensajes y ejemplos.

Para definir el protocolo de comunicación con el Directorio, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (que campos vienen a continuación). La operación dirdl no se muestra a continuación a pesar de ser un comando que interactúa con el directorio por el hecho de que este comando se implementara con mensajes binarios.

2.1.1.1 Tipos y descripción de los mensajes

Mensaje: ping
Sentido de la comunicación: Cliente -> Directorio
Descripción: contactar con el servidor de directorio para comprobar que está a la escucha y que utiliza un protocolo compatible con el del peer.
Ejemplo:
operation:pin\n
protocolid:12345678A\n

Mensaje: ping_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente es igual al que espera el servidor(Directorio).
Ejemplo:
operation:pin_ok\n

Mensaje:ping_error
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente no es igual al que espera el servidor(Directorio)
Ejemplo:
operation:ping_error\n

Mensaje: dirfiles
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide la lista de ficheros almacenados en el directorio.
Ejemplo:
operation:dirfiles\n

Mensaje: dirfiles_ok
Sentido de la comunicación: Cliente <- Directorio

Descripción: listado de ficheros donde para cada fichero se indica su nombre, tamaño y hash. Se tiene en cuenta que el listado puede acomodarse en más de un datagrama.

Ejemplo:

operation:dirfiles_ok\n
isLast:boolean\n
file:name,size,hash\n
file:name,size,hash\n
.
.
.
file:name,size,hash\n

Mensaje: serve
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide al directorio registrarse como un servidor de ficheros que escuche conexiones en un puerto TCP, y registrar dicha dirección de escucha (IP y puerto) en el directorio, asociándola al nickname del peer.

Ejemplo:

operation:server\n
serve:name,IP,puerto\n

Mensaje: serve_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: indica que se ha podido registrar como servidor ese peer

Ejemplo:

operation:serve_ok\n

Mensaje: serve_error
Sentido de la comunicación: Cliente <- Directorio
Descripción: indicar que no se ha podido registrar como servidor ese peer porque el nombre ya está registrado.

Ejemplo:

operation:serve_error\n

Mensaje: peers
Sentido de la comunicación: Cliente -> Directorio
Descripción: pide el censo de peers registrados como servidores en el directorio.

Ejemplo:

operation:peers\n

Mensaje: peers_ok
Sentido de la comunicación: Cliente <- Directorio
Descripción: muestra el censo de peers registrados como servidores en el directorio. El listado indica para cada par su nickname, IP y puerto TCP de escucha.

Ejemplo:

operation:serve_ok\n
serve:name,IP,puerto\n

serve:name,IP,puerto\n
.
.
.
serve:name,IP,puerto\n

Mensaje: quit
Sentido de la comunicación: Cliente -> Directorio
Descripción: cierra la aplicación. Si el servidor estaba activo, se detiene y se solicita la baja al directorio.
Ejemplo:
operation:quit\n

2.1.1.2 Código DirMessageOps

```
package es.um.redes.nanoFiles.udp.message;
public class DirMessageOps {
	/*
	 * TODO: (Boletín MensajesASCII) Añadir aquí todas las constantes que
definen
	 * los diferentes tipos de mensajes del protocolo de comunicación con
el
	 * directorio (valores posibles del campo "operation").
	 */
	public static final String OPERATION_INVALID = "invalid_operation";
	public static final String OPERATION_PING = "ping";
	public static final String OPERATION_PING_OK = "ping_ok";
	public static final String OPERATION_PING_ERROR = "ping_error";
	public static final String OPERATION_DIRFILES="dirfiles";
	public static final String OPERATION_DIRFILES_OK="dirfiles_ok";
	public static final String OPERATION_SERVE="serve";
	public static final String OPERATION_SERVE_OK="serve_ok";
	public static final String OPERATION_SERVE_ERROR="serve_error";
	public static final String OPERATION_PEERS="peers";
	public static final String OPERATION_PEERS_OK="peers_ok";
	public static final String OPERATION_QUIT="quit";

	// TODO: definir las operaciones del protocolo de directorio

}
```

2.1.1.3 Código DirMessage

```
package es.um.redes.nanoFiles.udp.message;

import java.net.InetSocketAddress;
import java.util.ArrayList;
import java.util.Collections;
```

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import es.um.redes.nanoFiles.util.FileInfo;

/**
 * Clase que modela los mensajes del protocolo de comunicación entre pares
para
 * implementar el explorador de ficheros remoto (servidor de ficheros). Estos
 * mensajes son intercambiados entre las clases DirectoryServer y
 * DirectoryConnector, y se codifican como texto en formato "campo:valor".
 *
 * @author rtitos
 *
 */
public class DirMessage {
    public static final int PACKET_MAX_SIZE = 65507; // 65535 - 8 (UDP
header) - 20 (IP header)

    private static final char DELIMITER = ':'; // Define el delimitador
    private static final char END_LINE = '\n'; // Define el carácter de fin
de línea

    /**
     * Nombre del campo que define el tipo de mensaje (primera línea)
     */
    private static final String FIELDNAME_OPERATION = "operation";
    private static final String FIELDNAME_PROTOCOL_ID="protocolid";
    private static final String FIELDNAME_FILE="file";
    private static final String FIELDNAME_LAST="last";
    private static final String FIELDNAME_SERVER="server";
    private static final String FIELDNAME_PEER="peer";

    /*
     * TODO: (Boletín MensajesASCII) Definir de manera simbólica los nombres
de
     * todos los campos que pueden aparecer en los mensajes de este protocolo
     * (formato campo:valor)
     */



    /**
     * Tipo del mensaje, de entre los tipos definidos en PeerMessageOps.
     */
    private String operation = DirMessageOps.OPERATION_INVALID;
    /**
     * Identificador de protocolo usado, para comprobar compatibilidad del
directorio.
     */
    private String protocolId;
    private boolean isLast = true;
```

```
    /*
     * TODO: (Boletín MensajesASCII) Crear un atributo correspondiente a cada
uno de
     * los campos de los diferentes mensajes de este protocolo.
     */

    private List<FileInfo> fileList;

    private String serverNickname;
    private InetSocketAddress serverAddress;

    private Map<String, InetSocketAddress> peerList;

    public DirMessage(String op) {
        operation = op;
        fileList=new ArrayList<>();
        peerList=new HashMap<>();
    }

    /*
     * TODO: (Boletín MensajesASCII) Crear diferentes constructores adecuados
para
     * construir mensajes de diferentes tipos con sus correspondientes
argumentos
     * (campos del mensaje)
     */




    public String getOperation() {
        return operation;
    }

    /*
     * TODO: (Boletín MensajesASCII) Crear métodos getter y setter para
obtener los
     * valores de los atributos de un mensaje. Se aconseja incluir código que
     * compruebe que no se modifica/obtiene el valor de un campo (atributo)
que no
     * esté definido para el tipo de mensaje dado por "operation".
     */
    public void setProtocolID(String protocolIdent) {
        if (!operation.equals(DirMessageOps.OPERATION_PING)) {
            throw new RuntimeException(
                    "DirMessage: setProtocolId called for message of
unexpected type (" + operation + ")");
        }
        protocolId = protocolIdent;
    }

    public String getProtocolId() {
```

```
        return protocolId;
    }

    public boolean addFile(FileInfo file) {
        return fileList.add(file);
    }

    public List<FileInfo> getFileList() {
        return Collections.unmodifiableList(fileList);
    }

    public void setLast( boolean last ) {
        this.isLast = last;
    }

    public boolean getLast() {
        return isLast;
    }

    public void setServerNickname(String sn) {
        serverNickname=sn;
    }

    public String getServerNickname() {
        return serverNickname;
    }

    public void setServerAddress(InetSocketAddress sa) {
        serverAddress=sa;
    }

    public InetSocketAddress getServerAddress() {
        return serverAddress;
    }

    public void addPeerList(String key, InetSocketAddress value) {
        peerList.put(key, value);
    }

    public Map<String, InetSocketAddress> getPeerList(){
        return Collections.unmodifiableMap(peerList);
    }
    /**
     * Método que convierte un mensaje codificado como una cadena de
caracteres, a
     * un objeto de la clase PeerMessage, en el cual los atributos
correspondientes
     * han sido establecidos con el valor de los campos del mensaje.
     *
     * @param message El mensaje recibido por el socket, como cadena de
caracteres
```

```
     * @return Un objeto PeerMessage que modela el mensaje recibido (tipo,
valores,
     *         etc.)
     */
    public static DirMessage fromString(String message) {
        /*
         * TODO: (Boletín MensajesASCII) Usar un bucle para parsear el
mensaje línea a
         * línea, extrayendo para cada línea el nombre del campo y el valor,
usando el
         * delimitador DELIMITER, y guardarlo en variables locales.
         */

        // System.out.println("DirMessage read from socket:");
        // System.out.println(message);
        String[] lines = message.split(END_LINE + "");
        // Local variables to save data during parsing
        DirMessage m = null;
        FileInfo file=null;
        InetSocketAddress address=null;

        String[] values;

        for (String line : lines) {
            int idx = line.indexOf(DELIMITER); // Posición del delimitador
            String fieldName = line.substring(0, idx).toLowerCase(); //
minúsculas
            String value = line.substring(idx + 1).trim();

            switch (fieldName) {
            case FIELDNAME_OPERATION: {
                assert (m == null);
                m = new DirMessage(value);
                break;
            }
            case FIELDNAME_PROTOCOL_ID: {
                m.setProtocolID(value);
                break;
            }
            case FIELDNAME_FILE: {

                values=value.split(",");

                if(values.length==3) {
                    file=new FileInfo(values[2], values[0],
Long.parseLong(values[1]), "");
                    m.addFile(file);
                }
                else {
                    throw new IllegalArgumentException("The message format is
incorrect");
                }
```

```
                break;
            }
            case FIELDNAME_LAST: {
                m.setLast(Boolean.parseBoolean(value));
                break;
            }
            case FIELDNAME_SERVER: {

                //TODO verificar que no se accedan a posiciones indebidas del
array

                values=value.split(",");

                if(values.length==3) {
                    m.setServerNickname(values[0]);
                    m.setServerAddress(new InetSocketAddress(values[1],
Integer.parseInt(values[2])));
                }
                else {
                    throw new IllegalArgumentException("The message format is
incorrect");
                }

                break;

            }
            case FIELDNAME_PEER: {

                values=value.split(",");

                if(values.length==3) {
                    address=new InetSocketAddress(values[1],
Integer.parseInt(values[2]));
                    m.addPeerList(values[0], address);
                }
                else {
                    throw new IllegalArgumentException("The message format is
incorrect");
                }

                break;
            }

            default:
                System.err.println("PANIC: DirMessage.fromString - message
with unknown field name " + fieldName);
                System.err.println("Message was:\n" + message);
                System.exit(-1);
            }
        }
```

```
        return m;
    }

    /**
     * Método que devuelve una cadena de caracteres con la codificación del
mensaje
     * según el formato campo:valor, a partir del tipo y los valores
almacenados en
     * los atributos.
     *
     * @return La cadena de caracteres con el mensaje a enviar por el socket.
     */
    public String toString() {

        StringBuffer sb = new StringBuffer();
        sb.append(FIELDNAME_OPERATION + DELIMITER + operation + END_LINE); //
Construimos el campo
        /*
         * TODO: (Boletín MensajesASCII) En función de la operación del
mensaje, crear
         * una cadena la operación y concatenar el resto de campos necesarios
usando los
         * valores de los atributos del objeto.
         */


        switch(operation) {
            case DirMessageOps.OPERATION_PING: {
                sb.append(FIELDNAME_PROTOCOL_ID + DELIMITER + protocolId +
END_LINE);
                break;
            }
            case DirMessageOps.OPERATION_DIRFILES_OK: {
                sb.append(FIELDNAME_LAST + DELIMITER + isLast + END_LINE);
                for(FileInfo file: fileList) {
                    sb.append(FIELDNAME_FILE + DELIMITER + file.fileName +
"," + file.fileSize + "," + file.fileHash + END_LINE);
                }
                break;
            }
            case DirMessageOps.OPERATION_SERVE: {
                sb.append(FIELDNAME_SERVER + DELIMITER + serverNickname + ","
+ serverAddress.getHostString() + "," + serverAddress.getPort() + END_LINE);
                break;
            }
            case DirMessageOps.OPERATION_PEERS_OK: {
                for(String peer : peerList.keySet()) {
                    sb.append(FIELDNAME_PEER + DELIMITER + peer + "," +
peerList.get(peer).getHostString() + "," + peerList.get(peer).getPort() +
END_LINE);
                }
```

```
                break;
            }

        }


        sb.append(END_LINE); // Marcamos el final del mensaje
        return sb.toString();
    }

}
```

2.1.1.4 Código Directory Connector

```
package es.um.redes.nanoFiles.udp.client;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.InetSocketAddress;
import java.net.SocketTimeoutException;
import java.util.Map;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import es.um.redes.nanoFiles.tcp.client.NFConnector;

import es.um.redes.nanoFiles.application.NanoFiles;
import es.um.redes.nanoFiles.udp.message.DirMessage;
import es.um.redes.nanoFiles.udp.message.DirMessageOps;
import es.um.redes.nanoFiles.util.FileInfo;

/**
 * Cliente con métodos de consulta y actualización específicos del directorio
 */
public class DirectoryConnector {
    /**
     * Puerto en el que atienden los servidores de directorio
     */
    private static final int DIRECTORY_PORT = 6868;
    /**
     * Tiempo máximo en milisegundos que se esperará a recibir una respuesta
por el
     * socket antes de que se deba lanzar una excepción
SocketTimeoutException para
     * recuperar el control
     */
    private static final int TIMEOUT = 1000;
    /**
     * Número de intentos máximos para obtener del directorio una respuesta a
una
     * solicitud enviada. Cada vez que expira el timeout sin recibir
respuesta se
     * cuenta como un intento.
```

```
     */
    private static final int MAX_NUMBER_OF_ATTEMPTS = 5;

    /**
     * Socket UDP usado para la comunicación con el directorio
     */
    private DatagramSocket socket;
    /**
     * Dirección de socket del directorio (IP:puertoUDP)
     */
    private InetSocketAddress directoryAddress;
    /**
     * Nombre/IP del host donde se ejecuta el directorio
     */
    private String directoryHostname;





    public static class DownloadedFile {
        public final String filename;
        public final long filesize;
        public final byte[] data;
        public final String filehash;

        public DownloadedFile(String filename, long fsize, byte[] data,
String filehash) {
            this.filename = filename;
            this.filesize = fsize;
            this.data = data;
            this.filehash = filehash;
        }
    }

    public DirectoryConnector(String hostname) throws IOException {
        // Guardamos el string con el nombre/IP del host
        directoryHostname = hostname;
        /* Boletin 2.1.1
        /*
         * TODO: (Boletín SocketsUDP) Convertir el string 'hostname' a
InetAddress y
         * guardar la dirección de socket (address:DIRECTORY_PORT) del
directorio en el
         * atributo directoryAddress, para poder enviar datagramas a dicho
destino.
         */
        /*
         * TODO: (Boletín SocketsUDP) Crea el socket UDP en cualquier puerto
para enviar
         * datagramas al directorio
         */
```

```
        directoryAddress=new
InetSocketAddress(InetAddress.getByName(hostname), DIRECTORY_PORT);

        socket=new DatagramSocket();


    }

    /**
     * Método para enviar y recibir datagramas al/del directorio
     *
     * @param requestData los datos a enviar al directorio (mensaje de
solicitud)
     * @return los datos recibidos del directorio (mensaje de respuesta)
     */
    private byte[] sendAndReceiveDatagrams(byte[] requestData) {
        byte responseData[] = new byte[DirMessage.PACKET_MAX_SIZE];
        byte response[] = null;
        if (directoryAddress == null) {
            System.err.println("DirectoryConnector.sendAndReceiveDatagrams:
UDP server destination address is null!");
            System.err.println(
                    "DirectoryConnector.sendAndReceiveDatagrams: make sure
constructor initializes field \"directoryAddress\"");
            System.exit(-1);

        }
        if (socket == null) {
            System.err.println("DirectoryConnector.sendAndReceiveDatagrams:
UDP socket is null!");
            System.err.println(
                    "DirectoryConnector.sendAndReceiveDatagrams: make sure
constructor initializes field \"socket\"");
            System.exit(-1);
        }
        /*
         * TODO: (Boletín SocketsUDP) Enviar datos en un datagrama al
directorio y
         * recibir una respuesta. El array devuelto debe contener únicamente
los datos
         * recibidos, *NO* el búfer de recepción al completo.
         */
        /*
         * TODO: (Boletín SocketsUDP) Una vez el envío y recepción asumiendo
un canal
         * confiable (sin pérdidas) esté terminado y probado, debe
implementarse un
         * mecanismo de retransmisión usando temporizador, en caso de que no
se reciba
         * respuesta en el plazo de TIMEOUT. En caso de salte el timeout, se
debe volver
         * a enviar el datagrama y tratar de recibir respuestas, reintentando
como
```

```
         * máximo en MAX_NUMBER_OF_ATTEMPTS ocasiones.
         */
        /*
         * TODO: (Boletín SocketsUDP) Las excepciones que puedan lanzarse al
         * leer/escribir en el socket deben ser capturadas y tratadas en este
método. Si
         * se produce una excepción de entrada/salida (error del que no es
posible
         * recuperarse), se debe informar y terminar el programa.
         */
        /*
         * NOTA: Las excepciones deben tratarse de la más concreta a la más
genérica.
         * SocketTimeoutException es más concreta que IOException.
         */

        DatagramPacket requestPacket=new DatagramPacket(requestData,
requestData.length, directoryAddress);
        DatagramPacket responsePacket=new DatagramPacket(responseData,
responseData.length);
        int intento = 1;
        boolean recibido = false;
        while ( intento <= MAX_NUMBER_OF_ATTEMPTS && !recibido ) {

            try {
                socket.send(requestPacket);

                socket.setSoTimeout(TIMEOUT);
                socket.receive(responsePacket);
                int respDataLen=responsePacket.getLength();
                response=new byte[respDataLen];
                System.arraycopy(responseData, 0, response, 0, respDataLen);
                recibido = true;

            }catch(SocketTimeoutException e) {
                System.err.println("Attempt " + intento + ": the server is
not responding, retrying...");
                if( intento == MAX_NUMBER_OF_ATTEMPTS ) {
                    System.err.println("Max retries reached. Giving up.");
                }
            }catch(IOException e) {
                System.err.println("Check your connection, cannot comunicate
with directory");
                break;
            }

            intento++;
        }

        if (response != null && response.length == responseData.length) {
            System.err.println("Your response is as large as the datagram
reception buffer!!\n"
```

```
                    + "You must extract from the buffer only the bytes that
belong to the datagram!");
        }
        return response;
    }

    /**
     * Método para probar la comunicación con el directorio mediante el envío
y
     * recepción de mensajes sin formatear ("en crudo")
     *
     * @return verdadero si se ha enviado un datagrama y recibido una
respuesta
     */
    public boolean testSendAndReceive() {
        /*
         * TODO: (Boletín SocketsUDP) Probar el correcto funcionamiento de
         * sendAndReceiveDatagrams. Se debe enviar un datagrama con la cadena
"ping" y
         * comprobar que la respuesta recibida empieza por "pingok". En tal
caso,
         * devuelve verdadero, falso si la respuesta no contiene los datos
esperados.
         */
        boolean success = false;

        byte[] request=new String("ping").getBytes();
        byte[] response=sendAndReceiveDatagrams(request);
        String respStr=new String(response);
        System.out.println("Contents of received datagram: "+respStr);
        success=(respStr.startsWith("pingok"));



        return success;
    }

    public String getDirectoryHostname() {
        return directoryHostname;
    }

    /**
     * Método para "hacer ping" al directorio, comprobar que está operativo y
que
     * usa un protocolo compatible. Este método no usa mensajes bien
formados.
     *
     * @return Verdadero si
     */
    public boolean pingDirectoryRaw() {
        boolean success = false;
        /*
         * TODO: (Boletín EstructuraNanoFiles) Basándose en el código de
```

```
         * "testSendAndReceive", contactar con el directorio, enviándole
nuestro
         * PROTOCOL_ID (ver clase NanoFiles). Se deben usar mensajes "en
crudo" (sin un
         * formato bien definido) para la comunicación.
         *
         * PASOS: 1.Crear el mensaje a enviar (String "ping&protocolId").
2.Crear un
         * datagrama con los bytes en que se codifica la cadena : 4.Enviar
datagrama y
         * recibir una respuesta (sendAndReceiveDatagrams). : 5. Comprobar si
la cadena
         * recibida en el datagrama de respuesta es "welcome", imprimir si
éxito o
         * fracaso. 6.Devolver éxito/fracaso de la operación.
         */

        byte[] request = new
String("ping&"+NanoFiles.PROTOCOL_ID).getBytes();
        byte[] response=sendAndReceiveDatagrams(request);
        String respStr=new String(response);
        if(respStr.equals("welcome")) {
            //TODO codigo para exito
            System.out.println("DirectoryConnector.pingDirectoryRaw:
operation successful");
            success=true;
        }
        else {
            //TODO codigo para fracaso
            System.out.println("DirectoryConnector.pingDirectoryRaw:
operation failed");
            success=false;
        }


        return success;
    }

    /**
     * Método para "hacer ping" al directorio, comprobar que está operativo y
que es
     * compatible.
     *
     * @return Verdadero si el directorio está operativo y es compatible
     */
    public boolean pingDirectory() {
        boolean success = false;
        /*
         * TODO: (Boletín MensajesASCII) Hacer ping al directorio 1.Crear el
mensaje a
         * enviar (objeto DirMessage) con atributos adecuados (operation,
etc.) NOTA:
```

```
         * Usar como operaciones las constantes definidas en la clase
DirMessageOps :
         * 2.Convertir el objeto DirMessage a enviar a un string (método
toString)
         * 3.Crear un datagrama con los bytes en que se codifica la cadena :
4.Enviar
         * datagrama y recibir una respuesta (sendAndReceiveDatagrams). :
5.Convertir
         * respuesta recibida en un objeto DirMessage (método
DirMessage.fromString)
         * 6.Extraer datos del objeto DirMessage y procesarlos 7.Devolver
éxito/fracaso
         * de la operación
         */

        DirMessage request=new DirMessage(DirMessageOps.OPERATION_PING);
        request.setProtocolID(NanoFiles.PROTOCOL_ID);
        byte[] requestString=request.toString().getBytes();
        byte[] responseString=sendAndReceiveDatagrams(requestString);
        DirMessage response=DirMessage.fromString(new
String(responseString));

        String operation = response.getOperation();

        switch (operation) {
            case DirMessageOps.OPERATION_PING_OK : {
                System.out.println("DirectoryConnector.pingDirectory:
operation successful");
                success=true;
                break;
            }
            case DirMessageOps.OPERATION_PING_ERROR : {
                System.out.println("DirectoryConnector.pingDirectory:
operation failed");
                success=false;
                break;
            }
        }


        return success;
    }

    /**
     * Método para dar de alta como servidor de ficheros en el puerto
indicado.
     *
     * @param serverPort El puerto TCP en el que este peer sirve ficheros a
otros
     * @return Verdadero si el directorio tiene registrado a este peer como
servidor
     *         y acepta la lista de ficheros, falso en caso contrario.
     */
```

```
    public boolean registerFileServer(int serverPort) {
        boolean success = false;

        // TODO: Ver TODOs en pingDirectory y seguir esquema similar

        DirMessage request=new DirMessage(DirMessageOps.OPERATION_SERVE);
        request.setServerNickname(NanoFiles.peerNickname);

request.setServerAddress((InetSocketAddress)socket.getLocalSocketAddress());
//TODO: usar serverPort y no eso
        byte[] requestString=request.toString().getBytes();
        byte[] responseString=sendAndReceiveDatagrams(requestString);
        DirMessage response=DirMessage.fromString(new
String(responseString));

        if(response.getOperation().equals(DirMessageOps.OPERATION_SERVE_OK))
{
            System.out.println("Serve succesful");
            success=true;
        }
        else {
            System.out.println("Serve failed");
            success=false;
        }


        return success;
    }

    /**
     * Método para obtener la lista de ficheros alojados en el directorio.
Para cada
     * fichero se debe obtener un objeto FileInfo con nombre, tamaño y hash.
     *
     * @return Los ficheros disponibles en el directorio, o null si el
directorio no
     *         pudo satisfacer nuestra solicitud
     */
    public FileInfo[] getFileList() {
        ArrayList<FileInfo> filelist = new ArrayList<>();
        // TODO: Ver TODOs en pingDirectory y seguir esquema similar
        boolean isLast = false;
        while( !isLast ) {
            DirMessage request=new
DirMessage(DirMessageOps.OPERATION_DIRFILES);
            byte[] requestString=request.toString().getBytes();
            byte[] responseString=sendAndReceiveDatagrams(requestString);
            DirMessage response=DirMessage.fromString(new
String(responseString));


if(response.getOperation().equals(DirMessageOps.OPERATION_DIRFILES_OK)) {
                filelist.addAll(response.getFileList());
```

```
                isLast = response.getLast();
            }else {
                isLast = true;
            }
        }
        return filelist.toArray(new FileInfo[0]);
    }

    public Map<String, InetSocketAddress> getPeerList() {
        Map<String, InetSocketAddress> peers = new LinkedHashMap<String,
InetSocketAddress>();

        DirMessage request=new DirMessage(DirMessageOps.OPERATION_PEERS);
        byte[] requestString=request.toString().getBytes();
        byte[] responseString=sendAndReceiveDatagrams(requestString);
        DirMessage response=DirMessage.fromString(new
String(responseString));

        if(response.getOperation().equals(DirMessageOps.OPERATION_PEERS_OK))
{
            System.out.println("Peers succesful");
            peers=response.getPeerList();
        }
        return peers;
    }

    public Map<String, InetSocketAddress[]> searchFilesByHash(String
hashSubstring) {
        Map<String, InetSocketAddress[]> results = new LinkedHashMap<String,
InetSocketAddress[]>();




        return results;
    }

    public DownloadedFile downloadFileFromDirectory(String hashSubstring) {
        byte[] fileData = null;
        String filename = null;
        long filesize = -1;
        String filehash = null;



        return new DownloadedFile(filename, filesize, fileData, filehash);
    }

    /**
     * Método para darse de baja como servidor de ficheros.
     *
     * @return Verdadero si el directorio tiene registrado a este peer como
servidor
```

```
     *         y ha dado de baja sus ficheros.
     */
    public boolean unregisterFileServer() {
        boolean success = false;




        return success;
    }




}
```

2.1.1.5 Código NFDirectoryServer

```
package es.um.redes.nanoFiles.udp.server;

import java.io.File;
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetSocketAddress;
import java.net.SocketException;
import java.util.LinkedHashMap;

import es.um.redes.nanoFiles.application.Directory;
import es.um.redes.nanoFiles.application.NanoFiles;
import es.um.redes.nanoFiles.udp.message.DirMessage;
import es.um.redes.nanoFiles.udp.message.DirMessageOps;
import es.um.redes.nanoFiles.util.FileInfo;
import es.um.redes.nanoFiles.util.NickGenerator;

public class NFDirectoryServer {

    private static final int MAX_MSG_SIZE_BYTES = 1024; //He cambiado esto
porque se cortaban los mensajes

    /**
     * Número de puerto UDP en el que escucha el directorio
     */
    public static final int DIRECTORY_PORT = 6868;

    /**
     * Socket de comunicación UDP con el cliente UDP (DirectoryConnector)
     */
    private DatagramSocket socket = null;
    /*
     * TODO: Añadir aquí como atributos las estructuras de datos que sean
necesarias
```

```
     * para mantener en el directorio cualquier información necesaria para la
     * funcionalidad del sistema nanoFilesP2P: ficheros alojados, servidores
     * registrados, etc.
     */
    /**
     * Lista de ficheros alojados en el directorio.
     */
    private FileInfo[] directoryFiles;
    /**
     * Lista de servidores registrados (IP, puerto TCP).
     */
    private LinkedHashMap<String, InetSocketAddress> registeredPeers;

    /**
     * Probabilidad de descartar un mensaje recibido en el directorio (para
simular
     * enlace no confiable y testear el código de retransmisión)
     */
    private double messageDiscardProbability;

    private void enviarPaquete( DirMessage m, InetSocketAddress addr) throws
IOException{
        String responseString = m.toString();
        byte[] responseData=responseString.getBytes();
        DatagramPacket responsePacket=new DatagramPacket(responseData,
responseData.length, addr);
        socket.send(responsePacket);
    }

    public NFDirectoryServer(double corruptionProbability, String
directoryFilesPath) throws SocketException {
        /*
         * Guardar la probabilidad de pérdida de datagramas (simular enlace
no
         * confiable)
         */
        messageDiscardProbability = corruptionProbability;
        /*
         * Cargar los ficheros del directorio compartido.
         */
        File dir = new File(directoryFilesPath);
        if (!dir.exists()) {
            dir.mkdirs();
        }
        directoryFiles = FileInfo.loadFilesFromFolder(directoryFilesPath);
        System.out.println("* Directory loaded " + directoryFiles.length + "
files from " + directoryFilesPath);
        /*
         * TODO: (Boletín SocketsUDP) Inicializar el atributo socket: Crear
un socket
         * UDP ligado al puerto especificado por el argumento directoryPort
en la
         * máquina local,
```

```
         */
        /*
         * TODO: (Boletín SocketsUDP) Inicializar atributos que mantienen el
estado del
         * servidor de directorio: peers registrados, etc.)
         */
        registeredPeers=new LinkedHashMap<>();

        socket=new DatagramSocket(DIRECTORY_PORT);

        if (NanoFiles.testModeUDP) {
            if (socket == null) {
                System.err.println("[testMode] NFDirectoryServer: code not
yet fully functional.\n"
                        + "Check that all TODOs in its constructor and 'run'
methods have been correctly addressed!");
                System.exit(-1);
            }
        }
    }

    public DatagramPacket receiveDatagram() throws IOException {
        DatagramPacket datagramReceivedFromClient = null;
        boolean datagramReceived = false;
        while (!datagramReceived) {
            /*
             * TODO: (Boletín SocketsUDP) Crear un búfer para recibir
datagramas y un
             * datagrama asociado al búfer (datagramReceivedFromClient)
             */
            /*
             * TODO: (Boletín SocketsUDP) Recibimos a través del socket un
datagrama
             */

            byte[] recvBuf = new byte[MAX_MSG_SIZE_BYTES];
            datagramReceivedFromClient=new DatagramPacket(recvBuf,
recvBuf.length);

            socket.receive(datagramReceivedFromClient);

            if (datagramReceivedFromClient == null) {
                System.err.println("[testMode]
NFDirectoryServer.receiveDatagram: code not yet fully functional.\n"
                        + "Check that all TODOs have been correctly
addressed!");
                System.exit(-1);
            } else {
                // Vemos si el mensaje debe ser ignorado (simulación de un
canal no confiable)
                double rand = Math.random();
                if (rand < messageDiscardProbability) {
                    System.err.println(
```

```
                            "Directory ignored datagram from " +
datagramReceivedFromClient.getSocketAddress());
                } else {
                    datagramReceived = true;
                }
            }

        }

        return datagramReceivedFromClient;
    }

    public void runTest() throws IOException {

        System.out.println("[testMode] Directory starting...");

        System.out.println("[testMode] Attempting to receive 'ping'
message...");
        DatagramPacket rcvDatagram = receiveDatagram();
        sendResponseTestMode(rcvDatagram);

        System.out.println("[testMode] Attempting to receive
'ping&PROTOCOL_ID' message...");
        rcvDatagram = receiveDatagram();
        sendResponseTestMode(rcvDatagram);
    }

    private void sendResponseTestMode(DatagramPacket pkt) throws IOException
{
        /*
         * TODO: (Boletín SocketsUDP) Construir un String partir de los datos
recibidos
         * en el datagrama pkt. A continuación, imprimir por pantalla dicha
cadena a
         * modo de depuración.
         */

        String receivedMessage=new String(pkt.getData(), 0, pkt.getLength());
        System.out.println(receivedMessage);

        /*
         * TODO: (Boletín SocketsUDP) Después, usar la cadena para comprobar
que su
         * valor es "ping"; en ese caso, enviar como respuesta un datagrama
con la
         * cadena "pingok". Si el mensaje recibido no es "ping", se informa
del error y
         * se envía "invalid" como respuesta.
         */

        String messageToClient;
        if(receivedMessage.equals("ping")) {
```

```
            messageToClient = new String("pingok");
        }
        else if(receivedMessage.startsWith("ping&")){

            String protocolID=receivedMessage.split("&")[1];
            String directoryProtocolID=NanoFiles.PROTOCOL_ID;

            if(protocolID.contentEquals(directoryProtocolID)) {

                messageToClient = new String("welcome");
            }
            else {

                messageToClient = new String("denied");
                }
        }
        else {
            System.err.println("Mensaje inválido");
            messageToClient = new String("invalid");
        }
        byte[] dataToClient = messageToClient.getBytes();
        InetSocketAddress clientAddr = (InetSocketAddress)
pkt.getSocketAddress();
        System.out.println(
                "Sending datagram with message \"" + messageToClient + "\"");
        System.out.println("Destination is client at addr: " + clientAddr);
        DatagramPacket packetToClient = new DatagramPacket(dataToClient,
dataToClient.length, clientAddr);
        socket.send(packetToClient);


        /*
         * TODO: (Boletín Estructura-NanoFiles) Ampliar el código para que,
en el caso
         * de que la cadena recibida no sea exactamente "ping", comprobar si
comienza
         * por "ping&" (es del tipo "ping&PROTOCOL_ID", donde PROTOCOL_ID
será el
         * identificador del protocolo diseñado por el grupo de prácticas
(ver
         * NanoFiles.PROTOCOL_ID). Se debe extraer el "protocol_id" de la
cadena
         * recibida y comprobar que su valor coincide con el de
NanoFiles.PROTOCOL_ID,
         * en cuyo caso se responderá con "welcome" (en otro caso, "denied").
         */

        String messageFromClient = new String(pkt.getData(), 0,
pkt.getLength());
        System.out.println("Data received: " + messageFromClient);
```

```
    }

    public void run() throws IOException {

        System.out.println("Directory starting...");

        while (true) { // Bucle principal del servidor de directorio
            DatagramPacket rcvDatagram = receiveDatagram();

            sendResponse(rcvDatagram);

        }
    }

    private void sendResponse(DatagramPacket pkt) throws IOException {
        /*
         * TODO: (Boletín MensajesASCII) Construir String partir de los datos
recibidos
         * en el datagrama pkt. A continuación, imprimir por pantalla dicha
cadena a
         * modo de depuración. Después, usar la cadena para construir un
objeto
         * DirMessage que contenga en sus atributos los valores del mensaje.
A partir de
         * este objeto, se podrá obtener los valores de los campos del
mensaje mediante
         * métodos "getter" para procesar el mensaje y consultar/modificar el
estado del
         * servidor.
         */

        String requestString=new String(pkt.getData(), 0, pkt.getLength());
        System.out.println(requestString);
        DirMessage request=DirMessage.fromString(requestString);



        /*
         * TODO: Una vez construido un objeto DirMessage con el contenido del
datagrama
         * recibido, obtener el tipo de operación solicitada por el mensaje y
actuar en
         * consecuencia, enviando uno u otro tipo de mensaje en respuesta.
         */
        String operation = request.getOperation(); // TODO: Cambiar!

        /*
         * TODO: (Boletín MensajesASCII) Construir un objeto DirMessage
(msgToSend) con
         * la respuesta a enviar al cliente, en función del tipo de mensaje
recibido,
         * leyendo/modificando según sea necesario el "estado" guardado en el
servidor
```

```
         * de directorio (atributos files, etc.). Los atributos del objeto
DirMessage
         * contendrán los valores adecuados para los diferentes campos del
mensaje a
         * enviar como respuesta (operation, etc.)
         */

        DirMessage response=new DirMessage(DirMessageOps.OPERATION_INVALID);

        switch (operation) {
        case DirMessageOps.OPERATION_PING: {

            if(request.getProtocolId().equals(NanoFiles.PROTOCOL_ID)) {
                System.out.println("Ping succesful");
                response=new DirMessage(DirMessageOps.OPERATION_PING_OK);
            }
            else {
                System.out.println("Ping failed");
                response=new DirMessage(DirMessageOps.OPERATION_PING_ERROR);
            }

            /*
             * TODO: (Boletín MensajesASCII) Comprobamos si el protocolId del
mensaje del
             * cliente coincide con el nuestro.
             */
            /*
             * TODO: (Boletín MensajesASCII) Construimos un mensaje de
respuesta que indique
             * el éxito/fracaso del ping (compatible, incompatible), y lo
devolvemos como
             * resultado del método.
             */
            /*
             * TODO: (Boletín MensajesASCII) Imprimimos por pantalla el
resultado de
             * procesar la petición recibida (éxito o fracaso) con los datos
relevantes, a
             * modo de depuración en el servidor
             */



            break;
        }
        case DirMessageOps.OPERATION_DIRFILES: {
                System.out.println("Dirfiles succesful");
                int totalFiles = directoryFiles.length;
                int chunkSize = 7;

                if( totalFiles == 0) {
                    response=new
DirMessage(DirMessageOps.OPERATION_DIRFILES_OK);
```

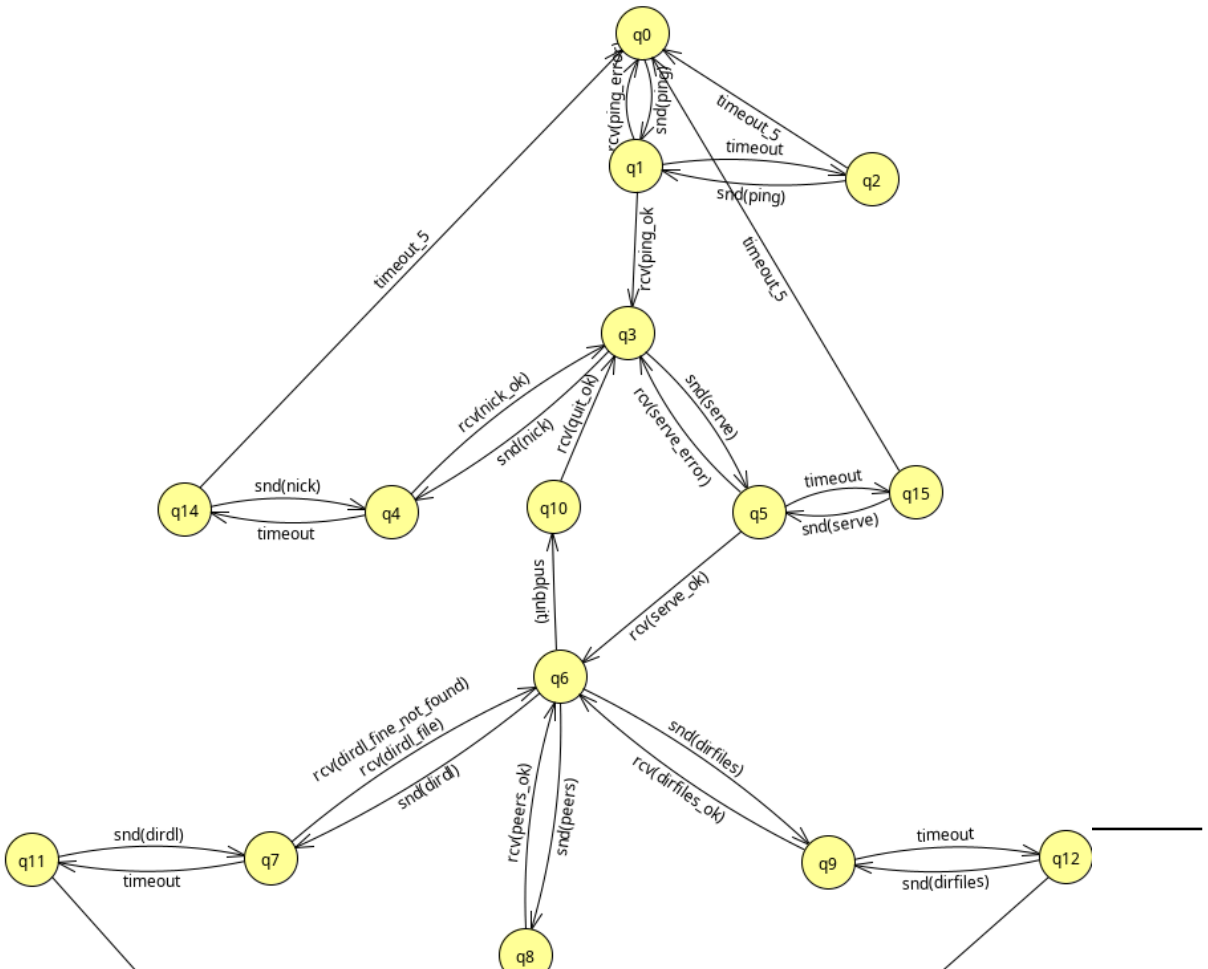
```
                    response.setLast(true);
                    enviarPaquete(response,
(InetSocketAddress)pkt.getSocketAddress());

                }else {
                    for(int i = 0; i < totalFiles; i += chunkSize) {
                        response = new
DirMessage(DirMessageOps.OPERATION_DIRFILES_OK);
                        int fin = Math.min(i + chunkSize, totalFiles);
                        for( int j = i; j < fin; j ++) {
                            response.addFile(directoryFiles[j]);
                        }
                        response.setLast(fin == totalFiles);
                        enviarPaquete(response,
(InetSocketAddress)pkt.getSocketAddress());
                    }
                }

            break;
        }
        case DirMessageOps.OPERATION_SERVE: {
            if(!registeredPeers.containsKey(request.getServerNickname())) {
                registeredPeers.put(request.getServerNickname(),
request.getServerAddress());
                response=new DirMessage(DirMessageOps.OPERATION_SERVE_OK);
            }
            else {
                response=new DirMessage(DirMessageOps.OPERATION_SERVE_ERROR);
            }

            break;
        }
        case DirMessageOps.OPERATION_PEERS: {
                response=new DirMessage(DirMessageOps.OPERATION_PEERS_OK);
                for(String peer : registeredPeers.keySet()) {
                    response.addPeerList(peer, registeredPeers.get(peer));
                }


            break;
        }

        default:
            System.err.println("Unexpected message operation: \"" + operation
+ "\"");
            System.exit(-1);
        }

        /*
         * TODO: (Boletín MensajesASCII) Convertir a String el objeto
DirMessage
         * (msgToSend) con el mensaje de respuesta a enviar, extraer los
bytes en que se
```

```
         * codifica el string y finalmente enviarlos en un datagrama
         */
        if(!operation.equals(DirMessageOps.OPERATION_DIRFILES_OK)) {
            String responseString=response.toString();
            byte[] responseData=responseString.getBytes();
            InetSocketAddress clientAddr = (InetSocketAddress)
pkt.getSocketAddress();
            DatagramPacket responsePacket=new DatagramPacket(responseData,
responseData.length, clientAddr);
            socket.send(responsePacket);
        }

    }



}
```

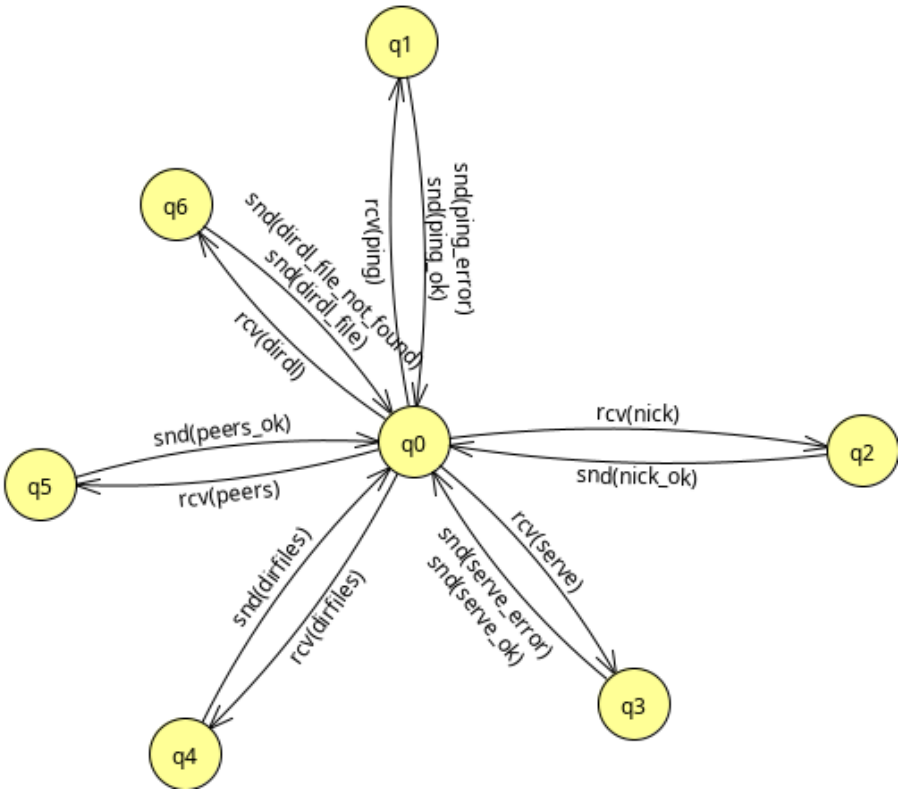
2.1.2 ***Autómatas.***

Estos son los esquemas de la lógica de los autómatas implementados en el proyecto.

2.1.2.1 Automata peer-directorio



2.1.2.2 Autómata directorio-peer



2.1.2.3 Codigo automatas

El código que se expone en esta sección es el mismo que para los autómatas que relacionan un peer con otro peer.

```
package es.um.redes.nanoFiles.logic;

import java.net.InetSocketAddress;

import es.um.redes.nanoFiles.application.NanoFiles;
import es.um.redes.nanoFiles.shell.NFCommands;
import es.um.redes.nanoFiles.shell.NFShell;
import es.um.redes.nanoFiles.util.FileInfo;

public class NFController {
    /**
     * Diferentes estados del cliente de acuerdo con el autómata
     */
    private static final byte OFFLINE = 0;
    /*
     * TODO: (Boletín Autómatas) Añadir más constantes que representen los estados
     * del autómata del cliente de directorio.
     */
    private static final byte PING = 1;
    private static final byte SERVE = 2;

    /**
     * Shell para leer comandos de usuario de la entrada estándar
     */
    private NFShell shell;
    /**
     * Último comando proporcionado por el usuario
     */
    private byte currentCommand;

    /**
     * Objeto controlador encargado de la comunicación con el directorio
     */
    private NFControllerLogicDir controllerDir;
    /**
     * Objeto controlador encargado de la comunicación con otros peers (como
     * servidor o cliente)
     */
    private NFControllerLogicP2P controllerPeer;

    /**
```

```
     * El estado en que se encuentra este peer (según el autómata). El estado debe
     * actualizarse cuando se produce un evento (comando) que supone un cambio en el
     * autómata.
     */
    private byte currentState;
    /**
     * Atributos donde se establecen los argumentos pasados a los distintos comandos
     * del shell. Estos atributos se establecen automáticamente según la orden y se
     * deben usar para pasar los valores de los parámetros a las funciones invocadas
     * desde este controlador.
     */
    private String targetHashSubstring; // Nombre del fichero a descargar
    private String targetPeerNickname; // Nickname para listar ficheros de un peer (peerfiles)
    private String newNickname; // Nuevo nickname solicitado por el usuario

    // Constructor
    public NFController(String defaultDirectory) {
        shell = new NFShell();

        String directory = shell.chooseDirectory(defaultDirectory);

        controllerDir = new NFControllerLogicDir(directory);
        controllerPeer = new NFControllerLogicP2P();
        // Estado inicial del autómata
        currentState = OFFLINE;

    }

    /**
     * Método que procesa los comandos introducidos por un usuario. Se encarga
     * principalmente de invocar los métodos adecuados de NFControllerLogicDir y
     * NFControllerLogicP2P según el comando.
     */
    public void testCommunication() {
        assert (NanoFiles.testModeUDP);
        System.out
                .println("[testMode] Attempting to reach directory server at " +
controllerDir.getDirectoryHostname());
        controllerDir.testCommunicationWithDirectory();
        System.out.println("[testMode] Test terminated!");
        return;
    }

    /**
     * Método que procesa los comandos introducidos por un usuario. Se encarga
     * principalmente de invocar los métodos adecuados de NFControllerLogicDir y
     * NFControllerLogicP2P según el comando.
     */
```

```
public void processCommand() {

    if (!canProcessCommandInCurrentState()) {
        return;
    }
    /*
     * En función del comando, invocar los métodos adecuados de NFControllerLogicDir
     * y NFControllerLogicP2P, ya que son estas dos clases las que implementan
     * realmente la lógica de cada comando y procesan la información recibida
     * mediante la comunicación con el directorio u otros pares de NanoFiles
     * (imprimir por pantalla el resultado de la acción y los datos recibidos,
     * etc.).
     */
    boolean commandSucceeded = false;
    switch (currentCommand) {
    case NFCommands.COM_MYFILES:
        showMyLocalFiles(); // Muestra los ficheros en el directorio local compartido
        break;
    case NFCommands.COM_PING:
        /*
         * Pedir al controllerDir enviar un "ping" al directorio, para comprobar que
         * está activo y disponible, y comprobar que es compatible.
         */
        commandSucceeded = controllerDir.ping();
        break;
    case NFCommands.COM_FILELIST_DIR:
        /*
         * Pedir al controllerDir que obtenga del directorio la lista de ficheros que
         * tiene, y la imprima por pantalla
         */
        controllerDir.getAndPrintFileList();
        break;
    case NFCommands.COM_PEERLIST:
        /*
         * Pedir al controllerDir que obtenga del directorio la lista de pares que están
         * sirviendo ficheros y la imprima por pantalla.
         */
        controllerDir.getAndPrintPeerList();
        break;
    case NFCommands.COM_FILELIST_PEER:
        /*
         * Pedir al controllerDir que obtenga del directorio la lista de pares que están
         * sirviendo ficheros, y luego pedir al controllerPeer que obtenga la lista de
         * ficheros que tiene disponible el peer dado por "targetPeerNickname"
         */
        java.util.Map<String, InetSocketAddress> peers = controllerDir.fetchPeerList();
        InetSocketAddress peerAddr = peers.get(targetPeerNickname);
        if (peerAddr == null) {
```

```
            System.err.println("* Peer '" + targetPeerNickname + "' not found in directory");
            break;
        }
        commandSucceeded = controllerPeer.listPeerFiles(peerAddr);
        break;
    case NFCommands.COM_DOWNLOAD_PEER:
        commandSucceeded = controllerPeer.downloadFromPeers(controllerDir,
targetPeerNickname,
                targetHashSubstring);
        break;
    case NFCommands.COM_SERVE:
        /*
         * Pedir al controllerPeer que lance un servidor de ficheros. Si el servidor se
         * ha podido iniciar correctamente, pedir al controllerDir darnos de alta como
         * servidor de ficheros en el directorio, indicando el puerto en el que nuestro
         * servidor escucha conexiones de otros peers así como la lista de ficheros
         * disponibles.
         */
        if (NanoFiles.testModeTCP) {
            controllerPeer.testTCPServer();
        } else {
            boolean serverRunning = controllerPeer.startFileServer();
            if (serverRunning) {
                commandSucceeded =
controllerDir.registerFileServer(controllerPeer.getServerPort());
            } else {
                System.err.println("Cannot start file server");
            }
        }
        break;
    case NFCommands.COM_DOWNLOAD_DIR:
        /*
         * Descargar directamente un fichero pequeño servido por el directorio (carpeta
         * dir-shared), identificado por subcadena de hash. Solo se descarga si el
         * tamaño cabe en un datagrama (lo asegura el servidor al responder).
         */
        commandSucceeded =
controllerDir.downloadAndSaveFromDirectory(targetHashSubstring);
        break;
    case NFCommands.COM_QUIT:
        /*
         * Pedir al controllerPeer que pare el servidor en segundo plano (método método
         * stopBackgroundFileServer). A continuación, pedir al controllerDir que
         * solicite al directorio darnos de baja como servidor de ficheros (método
         * unregisterFileServer).
         */
        if (controllerPeer.serving()) {
            controllerPeer.stopFileServer();
```

```
            commandSucceeded = controllerDir.unregisterFileServer();
        }
        break;
    case NFCommands.COM_NICK:
        if (controllerPeer.serving()) {
            System.err.println("* Cannot change nickname while serving files. Stop the server
first.");
            break;
        }
        if (newNickname == null || newNickname.isBlank()) {
            System.err.println("* Invalid nickname");
            break;
        }
        NanoFiles.peerNickname = newNickname;
        System.out.println("* Nickname changed to: " + NanoFiles.peerNickname);
        commandSucceeded = true;
        break;
    default:
    }

    updateCurrentState(commandSucceeded);
}

/**
 * Método que comprueba si se puede procesar un comando introducidos por un
 * usuario, en función del estado del autómata en el que nos encontramos.
 */
private boolean canProcessCommandInCurrentState() {
    /*
     * TODO: (Boletín Autómatas) Para cada comando tecleado en el shell
     * (currentCommand), comprobar "currentState" para ver si dicho comando es
     * válido según el estado actual del autómata, ya que no todos los comandos
     * serán válidos en cualquier estado. Este método NO debe modificar
     * clientStatus.
     */
    boolean commandAllowed = true;
    switch (currentCommand) {
    case NFCommands.COM_MYFILES: {
        commandAllowed = true;
        break;
    }
    case NFCommands.COM_PING: {
        if( currentState != OFFLINE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_NICK: {
```

```
        if ( currentState != PING) {
            commandAllowed = false;
        }
    }
    case NFCommands.COM_SERVE: {
        if ( currentState != PING) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_FILELIST_DIR: {
        if ( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_DOWNLOAD_DIR: {
        if ( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_PEERLIST: {
        if ( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_QUIT: {
        if( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_DOWNLOAD_PEER: {
        if( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }
    case NFCommands.COM_FILELIST_PEER: {
        if( currentState != SERVE) {
            commandAllowed = false;
        }
        break;
    }

    default:
```

```
            // System.err.println("ERROR: undefined behaviour for " + currentCommand +
"command!");
        }
        return commandAllowed;
    }

    private void updateCurrentState(boolean success) {
        /*
         * TODO: (Boletín Autómatas) Si el comando ha sido procesado con éxito, debemos
         * actualizar currentState de acuerdo con el autómata diseñado para pasar al
         * siguiente estado y así permitir unos u otros comandos en cada caso.
         */
        if (!success) {
            return;
        }
        switch (currentCommand) {
        case NFCommands.COM_PING: {
            currentState = PING;
            break;
        }
        case NFCommands.COM_SERVE: {
            currentState = SERVE;
            break;
        }
        case NFCommands.COM_QUIT: {
            currentState = PING;
            break;
        }
        default:
        }

    }

    private void showMyLocalFiles() {
        System.out.println("List of files in local folder:");
        FileInfo.printToSysout(NanoFiles.db.getFiles());
    }

    /**
     * Método que comprueba si el usuario ha introducido el comando para salir de la
     * aplicación
     */
    public boolean shouldQuit() {
        return currentCommand == NFCommands.COM_QUIT;
    }

    /**
     * Establece el comando actual
```

```
     *
     * @param command el comando tecleado en el shell
     */
    private void setCurrentCommand(byte command) {
        currentCommand = command;
    }

    /**
     * Registra en atributos internos los posibles parámetros del comando tecleado
     * por el usuario.
     */
    private void setCurrentCommandArguments(String[] args) {
        switch (currentCommand) {
        case NFCommands.COM_DOWNLOAD_DIR:
            targetHashSubstring = args[0];
            break;
        case NFCommands.COM_DOWNLOAD_PEER:
            targetPeerNickname = args[0];
            targetHashSubstring = args[1];
            break;
        case NFCommands.COM_FILELIST_PEER:
            targetPeerNickname = args[0];
            break;
        case NFCommands.COM_NICK:
            newNickname = args[0];
            break;
        default:
        }
    }

    /**
     * Método para leer un comando general
     */
    public void readGeneralCommandFromShell() {
        // Pedimos el comando al shell
        shell.readGeneralCommand();
        // Establecemos que el comando actual es el que ha obtenido el shell
        setCurrentCommand(shell.getCommand());
        // Analizamos los posibles parámetros asociados al comando
        setCurrentCommandArguments(shell.getCommandArguments());
    }

}
```

2.2. Peers.

2.2.1 Mensajes y ejemplos.

Mensaje: Peerfiles (opcode = 1)
Sentido de la comunicación: Servidor <- Cliente
Descripción: pide listar los ficheros disponibles en un peer servidor de ficheros dado por su nickname.
Ejemplo:

Opcode
(1 byte)
0x01

Mensaje: Peerfiles_ok (opcode = 2)
Sentido de la comunicación: Servidor -> Cliente
Descripción: indica que se han obtenido los ficheros del servidor de ficheros y los manda con el formato nombre, tamaño y hash.
Ejemplo:

Opcode	Número de ficheros	Longitud nombre	Nombre	Tamaño	Longitud hash	Hash
(1 byte)	(2 bytes)	(2 bytes)	(n bytes)	(2 bytes)	(2 bytes)	(n bytes)
0x02	0x00 0x01	0x00 0x02	ej	0x00 0x10	0x00 0x03	a3f

Mensaje: Peer_not_found (opcode = 3)
Sentido de la comunicación: Servidor -> Cliente
Descripción: indica que se el peer no se encuentra como servidor de ficheros.
Ejemplo:

Opcode
(1 byte)
0x03

Mensaje: Peerdl(opcode = 4)
Sentido de la comunicación: Servidor <- Cliente
Descripción: descargar un fichero de entre los ficheros disponibles en un peer dado por su nickname. El fichero será identificado por una subcadena del hash. Estas descargas se harán a través de chunks.

Ejemplo:

Opcode	Longitud hash	Hash	Posición leer	Tamaño chuck
(1 byte)	(2 bytes)	(n bytes)	(8 bytes)	(2 bytes)
0x04	0x00 0x02	ae	0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00	0x00 0x50

Mensaje: Peerdl_file (opcode = 5)
Sentido de la comunicación: Servidor -> Cliente
Descripción: devuelve el chuck pedido del fichero pedido a través de su hash.
Ejemplo:

Opcode	Tamaño chuck	Chuck
(1 byte)	(2 bytes)	(n bytes)
0x05	0x00 0x04	aff3

Mensaje: Peerdl_error (opcode = 6)
Sentido de la comunicación: Servidor -> Cliente
Descripción: indica que no se encuentra el fichero disponible en el servidor de ficheros
Ejemplo:

Opcode
(1 byte)
0x06

Mensaje: Peerdl* (opcode = 7)
Sentido de la comunicación: Servidor <- Cliente
Descripción: descargar un chuck de un fichero de entre los ficheros disponibles en los peers que tengan dicho fichero. El fichero será identificado por una subcadena del hash. El byte de comprobación indica si se está haciendo un primer barrido por todos los peers disponibles para comprobar si tienen el fichero que buscamos. O ,si por lo contrario, ya se está llevando a cabo los mensajes que contienen información de la descarga.
Ejemplo:

Opcode	Validez	Longitud hash	Hash	Posición leer	Tamaño chuck
(1 byte)	(1 byte)	(2 bytes)	(n bytes)	(8 bytes)	(2 bytes)
0x07	0x00	0x00 0x02	ae	0x00 0x00 0x00 0x00 0x00 0x00 0x00	0x00 0x50

				0x00	

Mensaje: Peerdl*_chuck (opcode = 8 y opcode = 9)
Sentido de la comunicación: Servidor -> Cliente
Descripción: devuelve el chuck pedido del fichero pedido a través de su hash. Cuando se devuelve un opcode = 8 es porque ya se está mandando información relativa a la descarga; sin embargo, un opcode = 9 es para indicar que el servidor de ficheros cuenta con el fichero buscado por el cliente.

Ejemplo:

Opcode	**Tamaño chuck**	**Chuck**
(1 byte)	**(2 bytes)**	**(n bytes)**
0x08	**0x00 0x04**	**aff3**

Opcode
(1 byte)
0x09

Mensaje: Peerdl_error (opcode = 10)
Sentido de la comunicación: Servidor -> Cliente
Descripción: indica que no se encuentra el fichero disponible en el servidor de ficheros

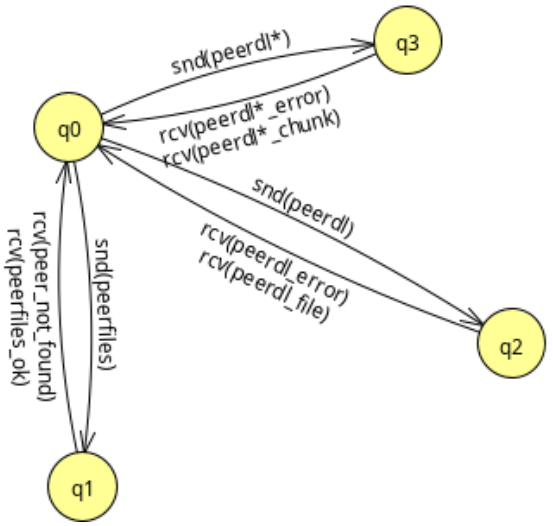
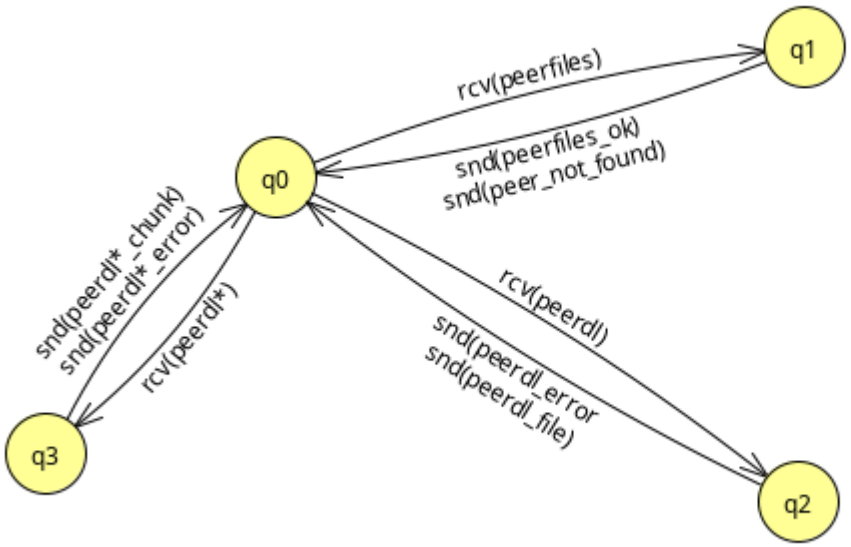
Ejemplo:

Opcode
(1 byte)
0x0A

2.2.2 Autómatas.

Solo se exponen las imágenes de la lógica de los autómatas porque el código de implementación es compartido con los autómatas anteriores.

2.2.2.1 Automata peer-peer-cliente



2.2.2.2 Automata peer-peer-servidor